

Contents

01 - Getting Started	2
Prerequisites	3
Build Instructions	3
Project Initialization	3
Your First Program	3
Writing Comments	4
02 - Variables and Types	4
Variable Declaration	4
Immutable Variables with @const	4
Type Annotations	4
Basic Types	5
String Operations	6
Escape Sequences	6
Reassignment Rules	7
Tuple Destructuring	7
03 - Operators	8
Arithmetic Operators	8
Comparison Operators	8
Logical Operators	9
Bitwise Operators	9
Compound Assignment Operators	9
Increment / Decrement Operators	10
Type Promotion Rules	10
Membership Operators	11
Control Flow	11
if / elif / else	11
while Loop	12
for Loop and range	12
break and continue	13
Nesting Example	13
Scope Rules	13
match	14
Functions	15
Basic Function Definition	15
Calling Functions	15
Recursive Functions	16
Function Overloading	16
Omitting the Return Type (Unit Type)	16
Structs and Enums	17
Defining Structs with record	17
Using Constructors	17
Field Access (Dot Notation)	17
Field Assignment	17
Structs as Function Parameters	17
Nested Structs	18
Enums	18
Collections	19

Tuples	19
Lists	20
Maps	22
Sets	23
Iterators	24
Advanced Features	24
Lambda Functions	24
Closures	25
Higher-Order Functions	25
Functions as Values	25
UFCS (Uniform Function Call Syntax)	26
Operator Overloading	26
Option Type	27
Concurrency Basics	27
Networking (TCP Sockets)	28
F-String (String Interpolation)	29
Type Casting (as)	29
Enum with Associated Data (ADT)	29
Generic Enum	30
Result Type	30
Packages	31
from/import Syntax	31
Subdirectories (Dot-Separated Paths)	31
Directory Packages	31
Standard Library (std)	31
Search Path Priority	32
RY_PATH Environment Variable	32
Limitations	32
Design by Contract	32
Preconditions (require)	33
Postconditions (ensure)	33
Combining require and ensure	33
Struct Invariants (invariant)	34
Rules	34
Testing	34
Running Tests	34
Writing Tests	34
Matchers	35
Output Format	35
Mocking	35
Parameterized Tests	36
Property-Based Tests	36
Limitations	36

[English](#) | [日本語](#) | [繁體中文](#)

01 - Getting Started

Next tutorial -> [02 - Variables and Types](#)

Prerequisites

To build and run Ry, you need the following:

- **LLVM 21**
 - **CMake 3.20 or later**
 - **A C++17 compatible compiler** (GCC 7+ / Clang 5+, etc.)
-

Build Instructions

Run the following commands from the repository root:

```
cmake -B build -DLLVM_DIR=/usr/local/llvm/lib/cmake/llvm
cmake --build build
```

On a successful build, an executable named `build/ry` will be generated.

Project Initialization

You can create a new project with the `ry new` command:

```
ry new my-project
cd my-project
```

This generates the following files and directories:

- `package.toml` – Project configuration file
- `src/main.ry` – Entry point (with sample code)

To initialize the current directory as a project instead, use `ry init`:

```
mkdir my-project
cd my-project
ry init
```

See [Project Management](#) for details.

Your First Program

Save the following content to a file named `hello.ry`:

```
print("Hello, World!")
```

Run it with the following command:

```
./build/ry hello.ry
```

Output:

```
Hello, World!
```

You can also run code from stdin using a pipe or here-document:

```
echo 'print("Hello, World!")' | ./build/ry
```

```
./build/ry <<'RY'
print("Hello, World!")
RY
```

Writing Comments

Everything from `#` to the end of the line is treated as a comment.

```
# This is a comment  
print("Hello") # End-of-line comments are also supported
```

Comments do not affect the behavior of the code.

Next tutorial -> [02 - Variables and Types](#)

[English](#) | [日本語](#) | [繁體中文](#)

02 - Variables and Types

<- [01 - Getting Started](#) / Next -> [03 - Operators](#)

Variable Declaration

In Ry, variables are declared using simple assignment syntax. By default, variables are mutable.

```
x = 42          # Inferred as int  
y = 3.14       # Inferred as float  
flag = true    # Inferred as bool  
name = "Ry"    # Inferred as str
```

Immutable Variables with `@const`

The `@const` directive marks a variable as immutable (constant). The value cannot be changed after declaration.

```
@const  
x = 42          # Inferred as int  
  
@const  
y = 3.14       # Inferred as float  
  
@const  
flag = true    # Inferred as bool  
  
@const  
name = "Ry"    # Inferred as str
```

Type Annotations

You can explicitly specify the type of a variable.

```
x: int = 42  
  
rate: float = 0.5
```

```
ok: bool = false
```

```
msg: str = "hello"
```

A compile error occurs if the type annotation does not match the actual type of the value.

Basic Types

Type	Description	Literal Examples
<code>int</code>	64-bit integer	0, 42, -10
<code>u8</code>	Unsigned 8-bit integer (0-255)	<code>b: u8 = 42</code>
<code>float</code>	64-bit floating-point number	0.0, 3.14, -1.5
<code>bool</code>	Boolean	<code>true</code> , <code>false</code>
<code>str</code>	String	"hello", ""

Low-Level Numeric Types

Ry also provides low-level numeric types for precise control over memory layout. These types have **no implicit conversions** — you must use `as` for explicit casts.

Type	Description	Example
<code>i8</code>	8-bit signed integer	<code>x: i8 = 42</code>
<code>i16</code>	16-bit signed integer	<code>x: i16 = 100</code>
<code>i32</code>	32-bit signed integer	<code>x: i32 = 42</code>
<code>i64</code>	64-bit signed integer	<code>x: i64 = 100</code>
<code>u8</code>	8-bit unsigned integer	<code>x: u8 = 200</code>
<code>u16</code>	16-bit unsigned integer	<code>x: u16 = 60000</code>
<code>u32</code>	32-bit unsigned integer	<code>x: u32 = 3000000000</code>
<code>u64</code>	64-bit unsigned integer	<code>x: u64 = 100</code>
<code>f32</code>	32-bit floating-point	<code>x: f32 = 3.14</code>

```
a: i32 = 10
b: i32 = 20
c = a + b           # OK: i32 + i32 → i32

d = 42
# e = a + d         # Error: cannot mix i32 and int

y = a as int        # Explicit cast to int
z = d as i32        # Explicit cast to i32

# Unsigned types use unsigned operations
x: u32 = 3000000000
y: u32 = 7
q = x / y           # Unsigned division (UDiv)
```

Note: `/` on low-level integers performs integer division (like Rust), not float division. Signed types use `SDiv`, unsigned types use `UDiv`.

Note: Arithmetic on low-level integers wraps on overflow. Signed types use two's complement, unsigned types use modular arithmetic. Use the high-level `int` type (64-bit) if overflow is a concern.

Fixed-Length Arrays

For low-level types, Ry provides fixed-length contiguous arrays `[T; N]`. These are stack-allocated with compile-time known size.

```
buf: [i32; 4] = [1, 2, 3, 4]
print(buf[0])      # 1
buf[2] = 99
print(buf[2])      # 99
print(length(buf)) # 4

pixels: [u8; 3] = [255, 128, 0]
```

String Operations

Various operations are available for strings.

```
a = "Hello"
b = "World"

# Concatenation
c = a + ", " + b    # "Hello, World"

# Comparison (lexicographic order)
print(a == b)      # false
print(a != b)      # true
print(a < b)       # true ("H" < "W")

# Length
print(length(a))   # 5

# Substring checks
s = "Hello, World!"
print(contains(s, "World"))    # true
print(starts_with(s, "Hello")) # true
print(ends_with(s, "!"))      # true
```

Escape Sequences

The following escape sequences can be used within strings.

Sequence	Meaning
<code>\n</code>	Newline
<code>\r</code>	Carriage return
<code>\t</code>	Tab
<code>\\</code>	Backslash
<code>\"</code>	Double quote
<code>\0</code>	Null character

```
print("Hello\nWorld")    # Outputs on two lines
print("A\tB")            # Tab-separated
print("say \"hi\"")      # String containing double quotes
```

Reassignment Rules

Variables declared without `@const` can be reassigned. However, the following restrictions apply:

```
x = 10
x = 20      # OK: reassignment to the same type
# x = "text" # Error: reassignment with a different type is not allowed
```

`@const` variables cannot be reassigned.

```
@const
N = 5
# N = 6 # Error: reassignment to a @const variable is not allowed
```

Redeclaring a variable with the same name is also not allowed.

```
x = 1
# x = 2 with another declaration in the same scope is not allowed
```

Tuple Destructuring

You can unpack a tuple into multiple variables in a single declaration.

```
@const
a, b = (10, 20)
print(a)    # 10
print(b)    # 20
```

Wildcard

Use `_` to ignore a position.

```
@const
x, _ = (1, 2)    # only x is bound; 2 is discarded
print(x)        # 1
```

Mutable Destructuring

Omit `@const` to declare mutable variables.

```
a, b = (10, 20)
a = 99
print(a)    # 99
```

Rules

- The number of variables on the left must match the number of elements in the tuple.
 - Each variable follows the same `@const`/mutable rules as a regular declaration.
 - Nested tuple destructuring is not supported.
-

[<- 01 - Getting Started](#) / [Next -> 03 - Operators](#)

[English](#) | [日本語](#) | [繁體中文](#)

03 - Operators

[<- 02 - Variables and Types](#) / Next -> [04 - Control Flow](#)

Arithmetic Operators

Operator	Description	Example	Result
+	Addition	3 + 2	5
-	Subtraction	3 - 2	1
*	Multiplication / string repetition	3 * 2	6
/	Division (always float)	7 / 2	3.5
//	Integer division (always int)	7 // 2	3
%	Modulo	7 % 3	1
**	Exponentiation (always float)	2 ** 10	1024.0

```
a = 10
b = 3

print(a + b)    # 13
print(a - b)    # 7
print(a * b)    # 30
print(a / b)    # 3.3333... (float)
print(a // b)   # 3 (int)
print(a % b)    # 1
print(2 ** 8)   # 256.0 (float)
```

Comparison Operators

All comparison operators return a bool value.

Operator	Description	Example
==	Equal to	a == b
!=	Not equal to	a != b
<	Less than	a < b
<=	Less than or equal to	a <= b
>	Greater than	a > b
>=	Greater than or equal to	a >= b

```
x = 5
y = 10

print(x == y)   # false
print(x != y)   # true
print(x < y)     # true
print(x <= y)    # true
print(x > y)     # false
print(x >= y)    # false
```

Comparison operators also work on strings (lexicographic comparison).


```
print("abc" == "abc")    # true
print("abc" < "abd")     # true
print("b" > "a")         # true
```

Logical Operators

Operator	Description	Example
and	Logical AND	a and b
or	Logical OR	a or b
not	Logical NOT	not a

```
t = true
f = false
```

```
print(t and f)    # false
print(t or f)     # true
print(not t)      # false
print(not f)      # true
```

Bitwise Operators

Bitwise operators can only be used with the `int` type.

Operator	Description	Example
&	Bitwise AND	5 & 3 -> 1
\	Bitwise OR	5 \ 3 -> 7
^	Bitwise XOR	5 ^ 3 -> 6
~	Bitwise NOT (unary)	~5 -> -6
<<	Left shift	1 << 3 -> 8
>>	Right shift (arithmetic)	8 >> 2 -> 2
>>>	Logical right shift	-1 >>> 1 -> 9223372036854775807

```
a = 0b1010    # 10
b = 0b1100    # 12
```

```
print(a & b)    # 8 (0b1000)
print(a | b)    # 14 (0b1110)
print(a ^ b)    # 6 (0b0110)
print(~a)       # -11
print(1 << 4)   # 16
print(32 >> 2)  # 8
```

Compound Assignment Operators

Shorthand notation for updating the value of a variable.

Operator	Description	Equivalent Expression
+=	Addition assignment	<code>x = x + n</code>
-=	Subtraction assignment	<code>x = x - n</code>
*=	Multiplication assignment	<code>x = x * n</code>
/=	Division assignment	<code>x = x / n</code>
%=	Modulo assignment	<code>x = x % n</code>

```
x = 10
x += 5    # x == 15
x -= 3    # x == 12
x *= 2    # x == 24
x /= 4    # x == 6.0 (becomes float)
```

Increment / Decrement Operators

Shorthand for incrementing or decrementing a variable by 1.

Operator	Description	Equivalent Expression
x++	Increment by 1	<code>x = x + 1</code>
x--	Decrement by 1	<code>x = x - 1</code>

```
count = 0
count++    # count == 1
count++    # count == 2
count--    # count == 1
```

Note: These are statement-only operators. They cannot be used inside expressions.

Type Promotion Rules

The following describes the behavior when `int` and `float` are mixed in operations.

```
# + - * produce float if either operand is float
print(1 + 2)      # 3 (int)
print(1 + 2.0)    # 3.0 (float)
print(1.0 + 2)    # 3.0 (float)

# / always produces float
print(4 / 2)      # 2.0 (float)

# // always produces int
print(7 // 2)     # 3 (int)
print(7.0 // 2)   # 3 (int)

# ** always produces float
print(2 ** 3)     # 8.0 (float)

# % produces int if both operands are int, float if either is float
print(7 % 3)      # 1 (int)
print(7.5 % 2)    # 1.5 (float)
```

```
# + concatenates strings if both operands are str
print("foo" + "bar")    # "foobar"

# * repeats a string if one operand is str and the other is int
print("ab" * 3)          # "ababab"
print(3 * "ab")          # "ababab"
```

Membership Operators

Operator	Description	Example
<code>in</code>	Membership test	<code>2 in {1, 2, 3} -> true</code>
<code>not in</code>	Negated membership test	<code>4 not in {1, 2, 3} -> true</code>

```
s = {1, 2, 3}
print(2 in s)      # true
print(4 not in s)  # true
```

[<- 02 - Variables and Types](#) / [Next -> 04 - Control Flow](#)

[English](#) | [日本語](#) | [繁體中文](#)

Control Flow

[<- Prev: Operators](#) | [Next: Functions ->](#)

if / elif / else

Use `if` to branch execution based on conditions.

```
x = 10
```

```
if x > 0:
    print(x)
elif x == 0:
    print(0)
else:
    print(-1)
```

- `elif` and `else` are optional.
- Conditions are not limited to `bool` values. For `int`, 0 is treated as `false` and non-0 as `true`.
- `if` statements can be nested.

```
a = 5
b = 3
```

```
if a > 0:
    if b > 0:
        print(a + b)    # 8
```

while Loop

Repeatedly executes a block as long as the condition is true.

```
i = 3
while i > 0:
    print(i)
    i = i - 1
# 3
# 2
# 1
```

for Loop and range

You can iterate over a list or using `range`.

```
for x in [1, 2, 3]:
    print(x)
# 1
# 2
# 3
```

`range(n)` generates integers from 0 to `n - 1`.

```
for i in range(5):
    print(i)
# 0
# 1
# 2
# 3
# 4
```

`range(start, end)` generates integers from `start` to `end - 1`.

```
for i in range(2, 5):
    print(i)
# 2
# 3
# 4
```

The `..` range operator creates an inclusive range: `1 .. 3` produces `[1, 2, 3]`.

```
for i in 1 .. 3:
    print(i)
# 1
# 2
# 3
```

You can iterate over map key-value pairs with `for k, v in map:`

```
m = {"x": 10, "y": 20}
for k, v in m:
    print(k)
    print(v)
```

break and continue

`break` immediately exits the loop. `continue` skips the current iteration and proceeds to the next one.

```
for i in range(10):
    if i == 5:
        break
    if i % 2 == 0:
        continue
    print(i)
# 1
# 3
```

They can also be used with `while` loops.

```
n = 0
while true:
    n = n + 1
    if n % 2 == 0:
        continue
    if n > 7:
        break
    print(n)
# 1
# 3
# 5
# 7
```

Note: In nested loops, `break` / `continue` only affect the innermost loop. Using them outside a loop results in a compile error.

Nesting Example

`for` and `while` loops can be nested.

```
for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            continue
        print(i * 10 + j)
# 11
# 13
# 21
# 23
# 31
# 33
```

Scope Rules

Control flow blocks have their own scope.

Block Scope

Variables declared inside a block cannot be referenced from outside the block.

```

if true:
    inner = 42
# Referencing inner here causes a compile error

```

Referencing and Reassigning Outer Variables

You can reference and reassign outer variables from within a block.

```

count = 0
for i in range(5):
    count = count + i
print(count)    # 10

```

Inner Scope Reassignment

Assigning to a variable inside a block modifies the outer variable (Python-style scoping). There is no shadowing — the inner assignment changes the same variable.

```

x = 1
if true:
    x = 99
    print(x)    # 99
print(x)        # 99

```

match

`match` is a construct for branching based on a value. It can safely handle enums and Options.

```

enum Color:
    Red
    Green
    Blue

c = Color::Green
match c:
    case Color::Red:
        print("red")
    case Color::Green:
        print("green")
    case Color::Blue:
        print("blue")
# green

```

Option Matching

Use `match` to safely handle both the `Some` and `None` cases.

```

x: Option<int> = Some(42)
match x:
    case Some(v):
        print(v)
    case None:
        print("nothing")
# 42

```

Wildcards and Literals

`_` is a wildcard pattern that matches anything. You can also match against literal values (numbers, strings, booleans).

```
n = 5
match n:
    case 0:
        print("zero")
    case 1:
        print("one")
    case _:
        print("other")
# other
```

Guard Clauses

You can add guard conditions with `if`.

```
match n:
    case x if x > 0:
        print("positive")
    case x if x < 0:
        print("negative")
    case _:
        print("zero")
```

Note: `match` must be exhaustive. For enums, all variants must be covered. For Options, both `Some` and `None` are required. For literals, a `_` wildcard is needed.

[<- Prev: Operators](#) | [Next: Functions ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Functions

[<- Prev: Control Flow](#) | [Next: Structs ->](#)

Basic Function Definition

Functions are defined with the `fn` keyword. Parameter type declarations are required and use the `name: type` format. The return type is specified after `->`.

```
fn add(a: int, b: int) -> int:
    return a + b
```

- Parameter type declarations are required.
 - The return type is specified after `->`.
 - Use `return` to return a value.
-

Calling Functions

Call a defined function by its name with arguments.

```
fn multiply(x: int, y: int) -> int:
    return x * y

result = multiply(3, 4)
print(result)    # 12
```

Recursive Functions

Functions can call themselves (recursion).

```
fn factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))    # 120
print(factorial(0))    # 1
```

Function Overloading

You can define multiple functions with the same name but different parameter counts or types.

```
fn add(a: int, b: int) -> int:
    return a + b

fn add(a: float, b: float) -> float:
    return a + b

print(add(1, 2))        # 3
print(add(1.5, 2.5))    # 4
```

The appropriate function is automatically selected based on the argument types at the call site.

Note: Defining functions with identical parameter types but different return types causes a compile error.

Omitting the Return Type (Unit Type)

If a function does not need to return a value, you can omit `->`. In this case, the function returns the Unit type.

```
fn greet():
    print(42)

greet()    # 42
```

This is the simplest form of a function with no parameters and no return value.

[<- Prev: Control Flow](#) | [Next: Structs ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Structs and Enums

[<- Prev: Functions](#) | [Next: Collections ->](#)

Defining Structs with record

Structs are defined with the `record` keyword. Each field is described in the `name: type` format.

```
record Point:  
  x: int  
  y: int
```

Structs are value types allocated on the stack.

Using Constructors

Create an instance by calling the struct name as a function. Arguments are specified in the order the fields are defined.

```
p = Point(10, 20)
```

Field Access (Dot Notation)

Fields are accessed using dot notation.

```
p = Point(10, 20)  
print(p.x)    # 10  
print(p.y)    # 20
```

Note: Passing a struct directly to `print` causes an error. Pass individual fields instead.

Field Assignment

Fields of mutable variables (declared without `@const`) can be reassigned.

```
p = Point(10, 20)  
p.x = 100  
print(p.x)    # 100
```

Note: Assigning to fields of a `@const` variable causes a compile error.

Structs as Function Parameters

Structs can be passed as function arguments.

```
record Point:  
  x: int  
  y: int  
  
fn distance_x(a: Point, b: Point) -> int:  
  return a.x - b.x
```

```
p1 = Point(10, 3)
p2 = Point(4, 7)
print(distance_x(p1, p2))    # 6
```

Nested Structs

A struct's field can be another struct.

```
record Point:
    x: int
    y: int

record Line:
    start: Point
    end: Point

line = Line(Point(0, 0), Point(10, 5))
print(line.start.x)    # 0
print(line.end.x)      # 10
```

You can access nested fields by chaining dot notation.

Enums

Enums are defined with the `enum` keyword. Each variant is treated as a named constant.

Definition

```
enum Color:
    Red
    Green
    Blue
```

Usage

Variants are accessed with `::`.

```
c = Color::Red
print(c)    # Red
```

Comparison

Variants can be compared using `==` and `!=`.

```
if c == Color::Red:
    print("red!")
elif c == Color::Green:
    print("green!")
else:
    print("blue!")
```

Function Parameters

Enum names can be used as function parameter types.

```
fn describe(c: Color) -> str:
  if c == Color::Red:
    return "warm"
  return "cool"
```

[<- Prev: Functions](#) | [Next: Collections ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Collections

[<- Prev: Structs](#) | [Next: Advanced Features ->](#)

Ry has four collection types: **Tuples**, **Lists**, **Maps**, and **Sets**.

Tuples

A tuple is an immutable data structure that groups multiple values together. It can hold elements of different types.

Creation

```
t = (1, 3.14)
```

Type Annotation

```
t: (int, float) = (1, 3.14)
```

Element Access

Elements are accessed by index using `.0`, `.1`, etc.

```
t = (1, 3.14)
print(t.0)    # 1
print(t.1)    # 3.14
```

As Function Return Values

Tuples are useful when you want to return multiple values.

```
fn swap(a: int, b: int) -> (int, int):
  return (b, a)
```

```
result = swap(1, 2)
print(result.0) # 2
print(result.1) # 1
```

Limitations

- Accessing an out-of-bounds index (e.g., `.2` on a tuple with 2 elements) causes a compile error.
 - Passing a tuple directly to `print` causes an error. Pass each element individually.
-

Lists

A list is a variable-length data structure containing elements of the same type.

Creation

```
xs = [1, 2, 3]
```

Type Annotation

```
xs: List<int> = [1, 2, 3]
```

Index Access

```
print(xs[0])    # 1
```

```
i = 1
print(xs[i])    # 2
```

Index Assignment

```
xs[0] = 99
```

length

```
print(length(xs))    # 3
```

print

```
print(xs)    # [1, 2, 3]
```

Iteration with for

```
for x in xs:
    print(x)
```

Function Parameters

```
fn first(xs: List<int>) -> int:
    return xs[0]
```

filter, map, sort

Lists support filter, map, and sort operations. These return new lists without modifying the original.

```
xs = [1, 2, 3, 4, 5]
```

```
# filter: keep elements matching a condition
evens = xs.filter(fn(x: int) => x > 3)
print(evens)    # [4, 5]
```

```
# map: transform each element
doubled = xs.map(fn(x: int) => x * 2)
print(doubled)    # [2, 4, 6, 8, 10]
```

```
# sort: sort in ascending order (default)
sorted = [3, 1, 2].sort()
print(sorted)    # [1, 2, 3]
```

```
# Chaining
result = xs.filter(fn(x: int) => x > 1).map(fn(x: int) => x * 10).sort()
print(result)    # [20, 30, 40, 50]
```

reduce, fold

`reduce` accumulates a list into a single value starting from the first element. `fold` does the same but with an explicit initial value.

```
xs = [1, 2, 3, 4, 5]

# reduce: start from first element
total = reduce(xs, fn(a: int, b: int) => a + b)
print(total)    # 15

# fold: provide an explicit initial value
total2 = fold(xs, 0, fn(a: int, b: int) => a + b)
print(total2)   # 15
```

any, all

`any` returns `true` if at least one element satisfies the predicate. `all` returns `true` if every element does.

```
xs = [1, 2, 3, 4, 5]

print(any(xs, fn(x: int) => x > 4))    # true
print(any(xs, fn(x: int) => x > 9))    # false

print(all(xs, fn(x: int) => x > 0))    # true
print(all(xs, fn(x: int) => x > 3))    # false
```

sum, min, max

```
xs = [3, 1, 4, 1, 5]
print(sum(xs))    # 14
print(min(xs))    # 1
print(max(xs))    # 5
```

first, last, is_empty

```
xs = [10, 20, 30]
print(first(xs))    # 10
print(last(xs))     # 30
print(is_empty(xs)) # false
```

enumerate, zip

`enumerate` pairs each element with its index. `zip` combines two lists element-by-element.

```
xs = [10, 20, 30]
indexed = enumerate(xs)
# [(0, 10), (1, 20), (2, 30)]
for p in indexed:
    print(p.0)
    print(p.1)
```

```
ys = ["a", "b", "c"]
zipped = zip(xs, ys)
# [(10, "a"), (20, "b"), (30, "c")]
```

Limitations

- All elements must be of the same type. Mixing different types is not allowed.
 - An empty list [] causes an error.
 - Out-of-bounds access results in a runtime error (`exit(1)`).
 - Supported element types are `int`, `float`, `bool`, and `str`.
-

Maps

A map is an associative array that manages key-value pairs.

Creation

```
m = {"a": 1, "b": 2}
```

Type Annotation

```
m: Map<str, int> = {"a": 1, "b": 2}
```

Key Access

```
print(m["a"])    # 1
```

Insertion / Update

Assigning to a new key inserts it; assigning to an existing key updates it.

```
m["c"] = 3        # Insert new entry
m["a"] = 99       # Update existing entry
```

length

```
print(length(m))  # 3
```

print

```
print(m)          # {a: 99, b: 2, c: 3}
```

has_key

Checks whether a key exists.

```
print(m.has_key("a"))  # true
```

keys, values

`keys` returns a list of all keys. `values` returns a list of all values.

```
m = {"a": 1, "b": 2, "c": 3}
print(keys(m))      # ["a", "b", "c"]
print(values(m))     # [1, 2, 3]
```

Function Parameters

```
fn get_val(m: Map<str, int>, k: str) -> int:  
    return m[k]
```

Limitations

- All keys must be of the same type, and all values must be of the same type.
 - An empty map requires a type annotation.
 - Accessing a nonexistent key results in a runtime error (`exit(1)`).
-

Sets

A set is a collection that holds elements of the same type without duplicates.

Creation

```
s = {1, 2, 3}
```

Type Annotation

```
s: Set<int> = {1, 2, 3}
```

in Operator

Use the `in` operator to check if an element is in the set.

```
print(2 in s)    # true  
print(5 in s)    # false
```

add / remove

```
s.add(4)         # Add element  
s.remove(1)      # Remove element  
s.add(2)         # Ignored since it already exists
```

length / print

```
print(length(s)) # 3  
print(s)         # {2, 3, 4}
```

Iteration with for

```
for x in s:  
    print(x)
```

Empty Set

An empty set requires a type annotation.

```
empty: Set<int> = {}
```

Limitations

- All elements must be of the same type.
 - Supported element types are `int`, `float`, `bool`, and `str`.
-

Iterators

Iterators provide a **lazy** way to process collections. Instead of creating intermediate lists at each step, iterators process elements one at a time through a pipeline.

Creating and Consuming

```
xs = [1, 2, 3]
ys = xs.iter().to_list()    # [1, 2, 3]
```

Chaining Operations

You can chain `filter`, `map`, and `take` to build pipelines:

```
result = [1, 2, 3, 4, 5]
    .iter()
    .filter(fn(x: int) => x > 2)
    .map(fn(x: int) => x * 2)
    .take(2)
    .to_list()
print(result)    # [6, 8]
```

Manual Iteration with next()

```
it = [10, 20].iter()
print(it.next())    # Some(10)
print(it.next())    # Some(20)
print(it.next())    # None
```

For Loops

Iterators work directly in `for` loops:

```
for x in [1, 2, 3].iter().filter(fn(x: int) => x > 1):
    print(x)    # 2, 3
```

Maps produce tuple elements:

```
for k, v in {"a": 1, "b": 2}.iter():
    print(k)
```

[<- Prev: Structs](#) | [Next: Advanced Features ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Advanced Features

[<- Prev: Collections](#) | [Next: Packages ->](#)

Lambda Functions

Lambda functions are a syntax for writing functions as expressions. They use the form `fn(parameters) => expression`. The return type is automatically inferred.

Single-Expression Lambda

```
double = fn(x: int) => x * 2
print(double(5)) # 10
```

```
add = fn(a: int, b: int) => a + b
print(add(3, 4)) # 7
```

No-Parameter Lambda

```
answer = fn() => 42
print(answer()) # 42
```

Multi-Line Lambda

You can write multiple statements by adding a newline after `:` and indenting.

```
abs = fn(x: int):
    if x < 0:
        return -x
    return x

print(abs(-5)) # 5
print(abs(3)) # 3
```

Closures

Lambda functions can capture variables from the scope in which they are defined.

```
offset = 10
add_offset = fn(x: int) => x + offset
print(add_offset(5)) # 15
```

Higher-Order Functions

You can define functions that take other functions as arguments. Function types are written as `fn(parameter_types) -> return_type`.

```
fn apply(f: fn(int) -> int, x: int) -> int:
    return f(x)

double = fn(x: int) => x * 2
print(apply(double, 3)) # 6
print(apply(fn(n: int) => n + 1, 10)) # 11
```

Functions as Values

Named functions can also be bound to variables or passed as arguments.

```
fn square(x: int) -> int:
    return x * x

fn apply(f: fn(int) -> int, x: int) -> int:
```

```

    return f(x)

# Pass a named function as an argument
print(apply(square, 4)) # 16

# Bind to a variable
sq = square
print(sq(5)) # 25

```

UFCS (Uniform Function Call Syntax)

With UFCS, you can write `f(a, b)` as `a.f(b)`. This enables method-chaining-style notation.

```

fn add(a: int, b: int) -> int:
    return a + b

x = 1
print(x.add(2)) # add(x, 2) -> 3

```

Chained Calls

```

fn double(n: int) -> int:
    return n * 2

print(x.add(2).double()) # double(add(x, 2)) -> 6

```

Operator Overloading

You can define operators for custom types using the `fn operator` syntax.

Binary Operators

Takes two parameters.

```

record Vec2:
    x: int
    y: int

fn operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)

fn operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

v1 = Vec2(1, 2)
v2 = Vec2(3, 4)
v3 = v1 + v2
print(v3.x) # 4
print(v1 == v2) # false

```

Unary Operators

Takes one parameter.

```
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

Supported Operators

Category	Operators
Arithmetic	+, -, *, /, %, **, //
Comparison	==, !=, <, <=, >, >=
Bitwise	&, \ , ^, ~, <<, >>
Logical	and, or, not

Option Type

A type that represents whether a value exists or not. It takes either `Some(value)` or `None`.

```
x: Option<int> = Some(42)
print(x)      # Some(42)

y: Option<int> = None
print(y)      # None
```

Extracting the Value

Use `match` to safely extract the inner value and handle the `None` case.

```
match x:
    case Some(v):
        print(v)      # 42
    case None:
        print("nothing")
```

Concurrency Basics

`Task<T>` is the runtime handle for concurrent work. Use `async fn` for task-returning functions, `await` inside another `async fn`, and `block_on(task)` from synchronous context to wait for completion.

```
async fn add(a: int, b: int) -> int:
    return a + b

# From synchronous context, use block_on()
t: Task<int> = add(20, 22)
print(block_on(t))          # 42
print(block_on(add(1, 2)))  # 3

# Inside async fn, use await
async fn double_add(a: int, b: int) -> int:
    return (await add(a, b)) * 2
```

`@parallel` can be applied to counted for loops over `range(...)` or integer .. ranges:

```
@parallel
for i in range(8):
    print(i)
```

@parallel for rejects break, continue, and writes to outer mutable variables.

Networking (TCP Sockets)

Ry provides TCP socket support through the `net` module. The `send`, `recv`, and `close` functions work with TCP sockets. All network operations return `Result` types.

```
from net import bind, listen, accept, connect, listener_port
from io import str_to_bytes, bytes_to_str

async fn echo_server(server: TcpListener) -> str:
    match accept(server):
        case Ok(conn):
            match recv(conn, 4096):
                case Ok(data):
                    match send(conn, data):
                        case Ok(_):
                            ...
                        case Err(e):
                            ...
                    case Err(e):
                        ...
            close(conn)
        case Err(e):
            ...
    close(server)
    return "done"

match bind("127.0.0.1", 0):
    case Ok(server):
        match listen(server, 1):
            case Ok(_):
                port = listener_port(server)
                t = echo_server(server)
                match connect("127.0.0.1", port):
                    case Ok(conn):
                        match send(conn, str_to_bytes("hello")):
                            case Ok(_):
                                ...
                            case Err(e):
                                ...
                        match recv(conn, 4096):
                            case Ok(resp):
                                match bytes_to_str(resp):
                                    case Ok(s):
                                        print(s)    # hello
                                    case Err(e):
                                        ...
                                case Err(e):
                                    ...
                        close(conn)
                    case Err(e):
                        print("connect failed")
```

```

        block_on(t)
    case Err(e):
        ...
    case Err(e):
        print("bind failed")

```

See [Network Reference](#) for the full API.

F-String (String Interpolation)

Use `f"..."` to embed expressions directly inside strings. Expressions are placed in `{}`.

```

name = "Alice"
print(f"Hello {name}")    # Hello Alice

x = 3
y = 4
print(f"{x} + {y} = {x + y}")    # 3 + 4 = 7

```

Use `{{` and `}}` to include literal braces.

```

print(f"{{escaped}}")    # {escaped}

```

Type Casting (as)

Convert between types explicitly with `as`.

```

x = 42 as float    # 42.0
y = 3.14 as int    # 3 (truncated)
s = 42 as str      # "42"
b = true as int    # 1

```

Enum with Associated Data (ADT)

Enum variants can carry associated values. This lets a single enum represent a family of different shapes of data. You can optionally name the fields for documentation clarity.

```

enum Shape:
    Circle(radius: float)
    Rectangle(width: float, height: float)
    Point

```

Named fields are documentation-only — they make definitions self-describing. Unnamed syntax (`Circle(float)`) is also valid.

Constructing ADT Variants

Construction is always positional, regardless of whether fields are named.

```

c = Shape::Circle(3.14)
r = Shape::Rectangle(4.0, 5.0)
p = Shape::Point

```

Matching ADT Variants

Use `case` with a binding pattern to extract the associated data. Bindings use your chosen variable names, not the field names.

```
fn describe(s: Shape) -> str:
  match s:
    case Shape::Circle(r):
      return f"circle with radius {r}"
    case Shape::Rectangle(w, h):
      return f"rectangle {w}x{h}"
    case Shape::Point:
      return "point"

print(describe(Shape::Circle(3.14)))      # circle with radius 3.14
print(describe(Shape::Rectangle(4.0, 5.0))) # rectangle 4.0x5.0
```

Generic Enum

Enums can take type parameters, making them reusable across different payload types.

```
enum MyOption<T>:
  MySome(T)
  MyNone
```

Usage

```
a = MyOption<int>::MySome(42)
b: MyOption<int> = MyOption<int>::MyNone
```

```
match a:
  case MyOption::MySome(v):
    print(v)      # 42
  case MyOption::MyNone:
    print("none")
```

Result Type

`Result<T, E>` is used for functions that may fail. Return `Ok(value)` for success and `Err(error)` for failure.

```
fn divide(a: int, b: int) -> Result<int, str>:
  if b == 0:
    return Err("division by zero")
  return Ok(a // b)
```

Use `match` to handle the result.

```
r = divide(10, 0)
match r:
  case Ok(v):
    print(v)
  case Err(e):
    print(e)      # division by zero
```

[<- Prev: Collections](#) | [Next: Packages ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Packages

[<- Prev: Advanced Features](#) | [Next: Design by Contract ->](#)

Ry uses a package system to organize code across files and directories. For the full specification, see [Package Reference](#).

from/import Syntax

Use the `from` syntax to import functions from another file.

```
from math import sqrt, PI    # Selective import
from math                   # Full import (all definitions)
```

This makes functions defined in `math.ry` available for use.

Subdirectories (Dot-Separated Paths)

Use dot-separated paths to specify packages in subdirectories.

```
from utils.calc import add    # Import from utils/calc.ry
```

Each dot corresponds to a directory separator.

Directory Packages

A package can be either a single `.ry` file or a directory containing multiple `.ry` files. When a package resolves to a directory, all `.ry` files in it are automatically loaded.

```
mypackage/
  calc.ry      # fn add(), fn sub()
  string.ry    # fn concat()

from mypackage          # imports add, sub, concat
from mypackage import add # imports only add
```

No special entry file (like `__init__.py`) is needed. Files starting with `_` are excluded.

Standard Library (std)

The `std` package is automatically imported into every program. You don't need to write `from std` — it's always available.

```
# These functions are available without any import
print("hello")
n = length("world")
xs = range(5)
```

You can also explicitly import specific definitions from standard library packages:

```
from str import contains
```

RY_HOME

The standard library is installed at `$RY_HOME/lib/std/`. The default value of `RY_HOME` is `~/.ry`.

```
export RY_HOME="$HOME/.ry" # default
```

Search Path Priority

Package files are searched in the following order:

1. **Directory of the importing file** — The directory containing the file with the import statement is searched first.
 2. **`$RY_HOME/lib`** — Standard library location.
 3. **Executable-relative `lib/`** — Directories relative to the `ry` executable.
 4. **`RY_PATH` environment variable** — If not found, directories specified in `RY_PATH` are searched in order.
-

RY_PATH Environment Variable

Multiple directories can be specified separated by colons.

```
export RY_PATH=/home/user/ry-libs:/usr/local/ry-libs
```

Once set, packages in the specified directories can be imported from anywhere.

Limitations

- `from` statements can only be written at the **top level** of a file. They cannot be placed inside functions or blocks.
- Importing the same package multiple times is automatically skipped (no duplicate imports).
- **Circular imports** (A imports B and B imports A) result in an error.

```
# Error example: a.ry and b.ry import each other
# a.ry: from b import foo
# b.ry: from a import bar <- circular import error
```

[<- Prev: Advanced Features](#) | [Next: Design by Contract ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Design by Contract

[<- Prev: Packages](#) | [Next: Testing ->](#)

Ry supports Eiffel-style Design by Contract with preconditions (**require**), postconditions (**ensure**), and struct invariants (**invariant**). Contract violations terminate the program. For the full specification, see [Design by Contract Reference](#).

Preconditions (**require**)

Use **require** to specify conditions that must be true when a function is called.

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  new_balance: int = balance + amount
  return new_balance
```

If any precondition fails, the program terminates with:

Contract violation: require failed in deposit()

Postconditions (**ensure**)

Use **ensure** to specify conditions that must be true when a function returns. The return value is bound to a user-chosen variable name.

```
fn abs(x: int) -> int:
  ensure v:
    v >= 0
  if x < 0:
    return -x
  return x
```

Since function arguments are immutable in Ry, you can reference them directly in **ensure** blocks:

```
fn increment(x: int) -> int:
  ensure v:
    v == x + 1
  return x + 1
```

For tuple returns, use multiple variable names separated by commas:

```
fn divide(a: int, b: int) -> (int, int):
  ensure q, r:
    q >= 0
    r >= 0
  return (a // b, a % b)
```

Combining **require** and **ensure**

Both can be used together. **require** must come before **ensure**.

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  ensure v:
    v >= 0
    v == balance + amount
  new_balance: int = balance + amount
  return new_balance
```

Struct Invariants (`invariant`)

Use `invariant` to specify conditions that must always hold for a struct. Invariants are checked after construction and after every field assignment.

```
record BankAccount:
  balance: int
  min_balance: int
  invariant:
    balance >= min_balance

a = BankAccount(100, 0)    # OK: 100 >= 0
# a.balance = -1          # Contract violation: invariant failed
```

Rules

- `require` and `ensure` blocks are optional and appear before the function body.
 - `require` must come before `ensure` when both are present.
 - `ensure` requires a variable binding (e.g., `ensure v:`) to name the return value.
 - For tuple returns, multiple bindings can be specified (e.g., `ensure q, r:`).
 - `invariant` appears at the end of a `record` definition, after all field declarations.
 - All contract violations terminate with `exit(1)`.
-

[<- Prev: Packages](#) | [Next: Testing ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Testing

[<- Prev: Design by Contract](#)

Ry has a built-in RSpec-style test syntax using `describe`, `it`, and `expect`. For the full specification, see [Testing Reference](#).

Running Tests

```
ry test                # Auto-discover and run all *.test.ry files
ry test tests/spec     # Run all *.test.ry files under a directory (recursive)
ry test tests/my_test.test.ry # Run a specific test file
ry test -p             # Run all tests in parallel (-p or --parallel)
```

The exit code is 0 if all tests pass, 1 if any test fails.

When run without arguments, `ry test` searches for `package.toml` to find the project root, then recursively discovers all `*.test.ry` files.

Writing Tests

Use `describe` to group related tests and `it` to define individual test cases.

```
describe("Calculator", fn():
  it("adds integers", fn():
```

```

        expect(1 + 2).to_eq(3)

    )
    it("subtracts integers", fn():
        expect(5 - 3).to_eq(2)

    )
    it("checks booleans", fn():
        expect(3 > 1).to_be_true()
    )
)

```

- `describe` and `it` take a description string and a **lambda argument** `fn()`: as the second parameter
- `describe`, `it`, `expect`, `mock`, and `verify` are only available with `ry test` (compile error with normal `ry` execution)

Matchers

Matcher	Description	Supported Types
<code>to_eq(expected)</code>	Equality comparison	int, float, bool, str
<code>to_not_eq(expected)</code>	Asserts not equal	int, float, bool, str
<code>to_be_true()</code>	Asserts <code>true</code>	bool
<code>to_be_false()</code>	Asserts <code>false</code>	bool
<code>to_be_none()</code>	Asserts <code>None</code>	Option
<code>to_be_some()</code>	Asserts Option is <code>Some</code>	Option
<code>to_contain(val)</code>	Asserts container includes value	List, Set, str

Output Format

Calculator

```

+ adds integers
+ subtracts integers
- checks booleans
  line 10: expected true, got false

```

2 passed, 1 failed

+ indicates pass (green), - indicates failure (red).

Mocking

`mock(fn_name, replacement)`

Replaces a function with a mock implementation for the current `it` block. The mock is automatically restored when the `it` block ends.

```

fn fetch_data() -> str:
    return "real data"

```

```

describe("mocking", fn():

```

```

    it("replaces function", fn():
        mock(fetch_data, fn() => "fake")
        expect(fetch_data()).to_eq("fake")
    )
    it("auto-restores", fn():
        expect(fetch_data()).to_eq("real data")
    )
)

```

verify(fn_name)

Returns the number of times a mocked function was called.

```

describe("verify", fn():
    it("counts calls", fn():
        mock(fetch_data, fn() => "fake")
        fetch_data()
        fetch_data()
        expect(verify(fetch_data)).to_eq(2)
    )
)

```

Parameterized Tests

Use `@each` to run the same test with multiple inputs:

```

@each([(1, 2, 3), (0, 0, 0), (-1, 1, 0)])
it("adds {0} + {1} = {2}", fn(a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
)

```

Each tuple becomes a separate test case. `{0}`, `{1}`, etc. in the description are replaced with actual values.

Property-Based Tests

Use `@property` to test with randomly generated inputs:

```

@property(count=100)
it("addition is commutative", fn(a: int, b: int):
    expect(a + b).to_eq(b + a)
)

```

The test runs `count` times with random values. On failure, the counterexample is printed.

Limitations

- Nesting of `describe` is not supported
- `before_each` / `after_each` are not supported
- Overloaded and `@native` functions cannot be mocked

[<- Prev: Design by Contract](#)